TheraLens: Comprehensive Objective Achievement Audit

Independent Scientific, Architectural, and Machine Learning Evaluation

Project Title: TheraLens: Machine Learning-Based Forecasting of Emerging Diseases and Future Therapeutic Development
Repository Directory: `D:\Projects\TheraLens`
Evaluation Role: Senior Machine Learning Research Auditor, Software Architect, Biomedical Data Analyst, & Scientific Validation Specialist
Milestone Target: Applied Machine Learning Comprehensive Objective Audit
Date of Audit: October 25, 2026
Audit Status: Complete, Evidence-Based, Non-Destructive

Table of Contents

1. Executive Summary
2. Original Objective and Acceptance Criteria
3. Repository and Architecture Overview
4. Dataset and Data-Lineage Audit
5. Stage-by-Stage Functional Audit
6. 5.1 Stage 1: Symptom Comparison vs. Disease Classification
7. 5.2 Stage 2: Therapeutic Evidence Investigation & Entity Linkage
8. 5.3 Stage 3: Therapeutic-Development Timeline & Cartesian Join Analysis
9. 5.4 Stage 4: Estimation of Medication Availability Future Year
10. 5.5 Stage 5: Modeling Advances in Medical & Technological Research
11. End-to-End Integration Audit
12. Machine Learning and Scientific Validation
13. 7.1 Classification Benchmark vs. Clinical Realities
14. 7.2 Disease-to-Medication Mapping Specificity
15. 7.3 Candidate Prioritization & Negative Control Analysis
16. 7.4 Epidemiological Time-Series Benchmarking
17. Future-Year Forecasting Audit (Mandatory Deep-Dive)
18. Testing and Reproducibility Audit
19. Requirements Achievement Scorecard
20. Prioritized Gap Analysis
21. Recommended Implementation Roadmap
22. Final Verdict & Core Question Answer

Section 1: Executive Summary

1.1 The Original Objective

The foundational objective of **TheraLens** is to investigate when an effective medication for a newly emerging disease might become available in the future. The intended scientific workflow connects: 1. Emerging disease symptom profile analysis and comparison against known reference diseases; 2. Retrieval of therapeutic evidence (targets, bioactive compounds, mechanisms of action) associated with the most similar known diseases; 3. Extraction of historical drug-development timelines, phase-transition latencies, and regulatory milestones; 4. Machine learning or empirical forecasting to project a realistic future calendar year for therapeutic availability; 5. Explicit computational accounting for the evolving rate of medical and technological discovery (e.g., AI screening, high-throughput assays, platform trials).

TheraLens was explicitly **never** intended to be a patient diagnosis tool, a clinical symptom checker, or a medical recommendation system. Its purpose is macro-level, translational research forecasting for emerging global health threats.

1.2 What the Current Implementation Actually Does

An exhaustive inspection of the source code (`src/`), data pipelines (`data/`), model artifacts (`models/`), generated outputs (`outputs/`), test suites (`tests/`), and execution logs reveals that TheraLens is currently a **collection of six decoupled, high-quality analytical modules executed sequentially by an orchestrator (`src/run_theralens.py`)**: 1. **Symptom Classifier (`src/models/train_symptom_classifier.py` , `src/inference/predict_disease.py`)**: Trains multinomial classification models (Logistic Regression, Random Forest, SVM) across a 132-symptom vocabulary to predict one of **41 pre-defined, closed-world disease labels**. It achieves 100% test accuracy on a deduplicated 304-pattern synthetic benchmark, but **does not compute symptom profile similarity** or handle open-world emerging diseases. 2. **Therapeutic Candidate Prioritization (`src/models/prioritize_therapeutic_candidates.py`)**: Computes a multi-attribute heuristic score (CPS) ranking 25,650 disease-target-molecule pairs across **37 pre-compiled known diseases** using Open Targets genomic association, ChEMBL median $p\text{ChEMBL}$ potency, Lipinski/Veber drug-likeness, and a clinical phase bonus. 3. **Epidemiological Trend Analysis (`src/analysis/analyze_epidemiological_trends.py`)**: Standardizes 5,426 longitudinal WHO surveillance records for Measles, Yellow Fever, and Cholera. Implements rolling Z-score anomaly detection and compares time-series baselines (Persistence, Moving Average, Autoregressive Ridge) on a 10-year test horizon. This module is **completely independent** of therapeutic timelines and symptom models. 4. **Regulatory Timeline Validation (`src/analysis/validate_therapeutic_timeline.py`)**: Harmonizes 12,243 Drugs@FDA approval records with ChEMBL molecule indications, resolving 3,059 of 3,142 ingredients (97.36% match rate). It audits an 18.25x Cartesian join expansion (555,323 rows representing 30,430 unique triplets) and separates Phase 4 approvals from Phase 1–3 investigational trials. 5. **Experimental Milestone Latency Projection (`src/analysis/forecast_therapeutic_timeline.py`)**: Computes milestone decision windows for the 37 pre-compiled diseases by taking the maximum clinical phase observed in ChEMBL and adding static, literature-derived DiMasi benchmark durations (Phase 1 = 2.1y, Phase 2 = 3.2y, Phase 3 = 3.8y, Review = 1.5y) to base year 2026. For 34 of 37 diseases (which already possess Phase 4 approved therapies), it outputs `2028` . **No machine learning regression model is trained or evaluated for future year prediction.** 6. **Model Explainability (`src/analysis/explain_model.py`)**: Generates permutation feature importances for the symptom classifier and score contribution breakdowns for top-ranked therapeutic candidates.

1.3 Key Audit Findings & Architectural Gaps

- **Gap 1: Classifier vs. Similarity Metric (Stage 1)**: The system implements a supervised multi-class classifier rather than an open-world similarity algorithm. When presented with symptoms of a truly *novel* disease, the classifier forces the input into one of 41 known training classes with an uncalibrated softmax margin, rather than quantifying degree of similarity, reporting overlapping vs. non-overlapping symptoms, or determining evidence sufficiency.
- **Gap 2: Complete Disconnection Between Inference and Therapeutics (Stage 2)**: The inference interface (`src/inference/predict_disease.py`) outputs a disease name and terminates. It does **not** call the therapeutic prioritization database, retrieve target associations, or list candidate molecules.
- **Gap 3: Complete Disconnection Between Inference and Forecasting (Stage 4)**: The inference interface outputs **no future year**. The forecasting script (`src/analysis/forecast_therapeutic_timeline.py`) is an offline batch script operating solely on 37 existing diseases; it cannot accept emerging disease inputs.
- **Gap 4: Absence of Machine Learning in Year Forecasting**: The future-year forecast is a deterministic arithmetic sum of hard-coded constants from a 2003/2016 paper. No predictive model is fitted to historical development latencies, and no historical temporal validation (backtesting) was performed on timelines.
- **Gap 5: Zero Implementation of Technological Progress (Stage 5)**: The model contains no parameters, features, or adjustments for AI-accelerated discovery, technological progress, or evolving clinical trial methodologies.

1.4 Readiness & Achievement Assessment

- **Component Engineering Readiness**: **HIGH (92%)**. The software engineering, data hygiene (deduplicating 4,961 synthetic rows to 304 unique patterns), FDA-ChEMBL entity resolution (97.36%), Cartesian disambiguation, testing (26/26 tests passing), and documentation are well-executed.
- **Central Research Objective Achievement**: **PARTIALLY ACHIEVED**. While sophisticated foundational subsystems have been constructed, the **unifying end-to-end scientific workflow is broken**. The system cannot take an emerging disease's symptoms, compute metric similarity, retrieve treatment evidence, and forecast a future medication availability year.

Section 2: Original Objective and Acceptance Criteria

To conduct an objective evaluation, the intended research workflow has been decomposed into nine verifiable acceptance criteria (AC_1 to AC_9).

ID	Intended Research Workflow Requirement	Verifiable Acceptance Criteria	Audit Verification Method
AC_1	**Symptom Profile Representation**	System represents clinical presentations as structured, comparable feature vectors covering standardized symptom vocabularies.	Verify vectorizer implementation, dimensionality, and support for missing/unseen terms.
AC_2	**Open-World Symptom Similarity**	System calculates explicit similarity metrics (e.g., Jaccard, Cosine, Hamming) between an arbitrary emerging disease profile and reference diseases, reporting similarity scores, overlapping symptoms, and differing symptoms.	Inspect whether similarity functions exist, or if only a closed-class classifier decision boundary is used.
AC_3	**Candidate Disease Retrieval**	System dynamically retrieves the top-k most similar reference diseases based on metric similarity thresholds, flagging cases where similarity is insufficient.	Trace control flow from symptom input to multi-disease similarity ranking.
AC_4	**Dynamic Therapeutic Evidence Linkage**	System automatically links the identified similar diseases to their associated biological targets, bioactive molecules, $p\text{ChEMBL}$ affinities, and mechanisms of action.	Inspect code connecting Stage 1 disease outputs to Stage 2 candidate prioritization tables.
AC_5	**Development History & Milestone Extraction**	System retrieves historical development records (preclinical discovery, IND filing, Phase 1–3 transitions, FDA approval years) for retrieved therapeutic candidates.	Verify data structure of `therapeutic_development_timeline.csv` and whether chronological transition intervals are computed.
AC_6	**Predictive Future-Year Forecasting**	System uses empirical data and predictive algorithms (machine learning regression, survival analysis, or probabilistic state transitions) to forecast a specific future medication availability year for the emerging disease.	Inspect forecasting algorithm: evaluate target variable, input features, training set, and prediction mechanism.
AC_7	**Technological Progress Modeling**	System computationally accounts for advances in biotechnology, AI screening, and clinical efficiency through time-dependent trend variables, acceleration parameters, or scenario bounds.	Inspect mathematical formulas and code for dynamic technology/efficiency adjustment factors.
AC_8	**Forecast Validation & Backtesting**	Forecasting model is evaluated against historical drug approvals via chronological train/test splits (e.g., train on pre-2010 approvals, forecast 2011–2025 outcomes) and reports standard error metrics (MAE, RMSE, Coverage).	Check for historical milestone validation routines and empirical accuracy benchmarks.
AC_9	**End-to-End Pipeline Cohesion**	A single user-facing entry point accepts an emerging disease symptom profile and generates a coherent report detailing similar diseases, treatment evidence, milestone forecasts, and scientific caveats.	Trace CLI and orchestrator execution from symptom input to final forecast output.

Section 3: Repository and Architecture Overview

3.1 Repository Layout

The workspace at `D:\Projects\TheraLens` is organized into a modular Python project structure:

```
D:\Projects\TheraLens\
├── data/
│   ├── raw/                            # Original source datasets
│   │   ├── symptom_disease.csv         # 4,961 raw symptom rows (41 classes, 132 symptoms)
│   │   ├── therapeutic_associations.csv  # 3,636 Open Targets gene-disease associations
│   │   ├── chembl_activity.csv         # 8,023 curated bioactivity assay records
│   │   ├── chembl_molecules.csv        # 6,319 curated small-molecule structures & properties
│   │   ├── measles_who.csv             # 1,692 WHO longitudinal measles surveillance records
│   │   ├── yellow_fever_who.csv        # 874 WHO annual yellow fever surveillance records
│   │   ├── cholera_2012..2014_who.csv  # 55 WHO weekly cholera surveillance records (wide format)
│   │   └── fda/                        # 12 Drugs@FDA raw tables (Products, Submissions, etc.)
│   └── processed/                      # Normalized, harmonized pipeline artifacts
│       ├── symptom_disease_clean.csv   # 304 deduplicated unique symptom vectors
│       ├── disease_target_rankings.csv # 3,636 ranked disease targets
│       ├── therapeutic_candidate_rankings.csv # 25,650 ranked candidate pairs (37 diseases)
│       ├── epidemiological_trends.csv  # 5,426 standardized long-format WHO records
│       ├── fda_drug_approval_timeline.csv# 12,243 normalized FDA product approvals
│       ├── fda_chembl_match_audit.csv  # 3,142 audited ingredient-to-ChEMBL resolutions
│       └── therapeutic_development_timeline.csv # 555,323 master joined regulatory rows
├── models/
│   ├── symptom_classifier.joblib       # Serialized Logistic Regression pipeline (44.2 KB)
│   ├── symptom_classifier_metadata.json  # Training parameters, metrics, class labels (1.4 KB)
│   ├── feature_names.json              # 132 symptom feature names (2.8 KB)
│   └── class_labels.json               # 41 target disease class names (0.8 KB)
├── outputs/                            # 52 production analytical artifacts (CSV, PNG, JSON, TXT)
│   ├── model_evaluation/               # Confusion matrix, model comparison metrics
│   ├── therapeutic_prioritization/     # Candidate score distributions, diagnostic charts
│   ├── epidemiological_analysis/       # Surge detection, 10-year forecast comparisons
│   ├── timeline_analysis/              # Regulatory audit, Cartesian expansion, milestone forecasts
│   └── explainability/                 # Permutation importance, score decompositions
├── src/                                # Source implementation code (11 core scripts)
│   ├── run_theralens.py                # Master pipeline orchestrator
│   ├── inference/predict_disease.py    # Symptom-based disease inference CLI
│   ├── models/                         # Training, evaluation, and candidate ranking
│   └── analysis/                       # Epidemiology, timeline validation, forecasting, explainability
├── tests/                              # 5 test modules containing 26 passing automated tests
└── docs/                               # Documentation, reports, and build scripts
```

3.2 Actual Architecture Diagram

The diagram below illustrates the actual data and control flow implemented in the codebase:

```
[ Raw Datasets ]
  ├── symptom_disease.csv ──────────> Deduplication (4,961 -> 304) ──> Train Classifier (Logistic Regression)
  │                                                                              │
  │                                                                              ▼
  │                                                                   models/symptom_classifier.joblib
  │                                                                              │
  │   [User Symptom Query] ──────────────────────────────────────────────────> CLI Inference Engine
  │                                                                             (predict_disease.py)
  │                                                                              │
  │                                                                              ▼
  │                                                                   Predicted Disease Class (1 of 41)
  │                                                                   [WORKFLOW TERMINATES HERE]
  │
  ├── therapeutic_associations.csv ─┐
  ├── chembl_activity.csv ──────────┼─> Target & Molecule Inner Join ─> Multi-Criteria Scoring (CPS)
  └── chembl_molecules.csv ─────────┘   (target_symbol, molecule_id)          │
                                                                              ▼
                                                                   therapeutic_candidate_rankings.csv
                                                                   (25,650 candidates across 37 diseases)
                                                                              │
  ├── Drugs@FDA Archive ────────────┐                                         ▼
  ├── chembl_drug_indications.csv ──┼─> Cartesian Integration Join ──> Milestone Latency Projection
  └── fda_chembl_match_audit.csv ───┘   (18.25x Expansion, 555k rows) (forecast_therapeutic_timeline.py)
                                                                              │
                                                                              ▼
                                                                   experimental_forecast_projections.csv
                                                                   (37 Static DiMasi Benchmark Projections)

  ├── measles_who.csv ──────────────┐
  ├── yellow_fever_who.csv ─────────┼─> Long Format Standardization ──> Rolling Z-Score Surge Detection
  └── cholera_2012..2014_who.csv ───┘   (5,426 longitudinal rows)        & Naive vs Ridge Forecasting
                                                                         [STANDALONE SILO]
```

Section 4: Dataset and Data-Lineage Audit

A comprehensive audit of all raw and processed data artifacts was performed by inspecting row counts, schema dimensions, null semantics, and join behaviors directly from disk.

Dataset Identifier	Physical Path	Source / Authority	Ingested Dimensions	Cleaned / Processed Dimensions	Primary Keys / Join Identifiers	Data Hygiene & Preprocessing Quality
Symptom Manifold	`data/raw/symptom_disease.csv`	Synthetic Benchmark Archive	4,961 $\times$ 133	304 $\times$ 133 (`symptom_disease_clean.csv`)	`prognosis` (Target), 132 binary symptom flags	**Excellent**: Audited 4,657 redundant synthetic duplicates (93.87% duplication rate). Deduplicated to 304 unique patterns. Zero pattern intersection between train and test splits.
Open Targets Associations	`data/raw/therapeutic_associations.csv`	Open Targets GraphQL API	3,636 $\times$ 7	3,636 $\times$ 8 (`disease_target_rankings.csv`)	`local_disease` , `disease_id` (EFO/MONDO), `target_symbol` , `target_id`	**High**: Association scores bounded $[0.0, 1.0]$. Curated for 37 target disease entities.
ChEMBL Bioactivity	`data/raw/chembl_activity.csv`	EMBL-EBI ChEMBL API	8,023 $\times$ 10	25,650 cross-referenced pairs	`target_symbol` , `chembl_target_id` , `molecule_chembl_id` , `pchembl_value`	**High**: Filtered from 22,238 raw query records. Median $p\text{ChEMBL}$ computed per target-molecule pair to eliminate assay bias.
ChEMBL Molecules	`data/raw/chembl_molecules.csv`	EMBL-EBI ChEMBL API	6,319 $\times$ 11	6,319 $\times$ 11	`molecule_chembl_id` , `pref_name` , `max_phase` , physicochemical props	**High**: Filtered from 8,983 candidate compounds. Molecular weight, ALogP, PSA, HBA, HBD parsed for Lipinski/Veber rule evaluation.
FDA Approval Timeline	`data/processed/fda_drug_approval_timeline.csv`	FDA Drugs@FDA Archive	12,243 $\times$ 12	12,243 $\times$ 12	`appl_no` , `active_ingredient` , `first_approval_year`	**High**: Synthesized from 12 Drugs@FDA tables. Spans 1939–2026. 749 records lack explicit approval dates (retained as null).
FDA-ChEMBL Match Audit	`data/processed/fda_chembl_match_audit.csv`	TheraLens Resolution Engine	3,142 $\times$ 9	3,142 $\times$ 9	`fda_active_ingredient` , `normalized_active_ingredient` , `selected_chembl_id`	**Rigorous**: Exactly 3,142 unique FDA active ingredients audited. 3,059 matched (97.36%) across 5 tiers; 83 unmatched logged.
ChEMBL Indications	`data/processed/chembl_drug_indications.csv`	EMBL-EBI ChEMBL API	30,663 $\times$ 8	30,663 $\times$ 8	`molecule_chembl_id` , `efo_id` , `mesh_id` , `max_phase_for_ind`	**Moderate**: Maps compound indications to MeSH/EFO disease ontologies.
Master Development Timeline	`data/processed/therapeutic_development_timeline.csv`	TheraLens Integration	555,323 $\times$ 14	30,430 unique triplets	`disease_name` , `chembl_molecule_id` , `max_phase_for_ind` , `first_approval_year`	**Audited**: 555,323 rows result from an 18.25x Cartesian join expansion. Separates 72,277 verified Phase 4 records from 483,043 investigational records.
WHO Surveillance	`data/raw/measles_who.csv` , `yellow_fever_who.csv` , `cholera_2012..2014_who.csv`	World Health Organization	2,621 combined rows	5,426 $\times$ 11 (`epidemiological_trends.csv`)	`disease` , `country_iso3` , `year` , `time_period` , `reported_cases`	**High**: Melted 52 weekly cholera columns into normalized long format. Retained unrecorded surveillance values as NaN (zero data distortion).

Section 5: Stage-by-Stage Functional Audit

5.1 Stage 1: Symptom Comparison vs. Disease Classification

- **Intended Requirement**: Accept symptom profiles of a newly emerging disease, compare against known reference diseases using similarity metrics, output degree of similarity, identify overlapping and differing symptoms, and assess evidence sufficiency.
- **Actual Implementation**:
- Implemented in `src/models/train_symptom_classifier.py` and `src/inference/predict_disease.py`.
- Feature representation: A 132-dimensional binary vector aligned with `models/feature_names.json`.
- Algorithm: Multinomial Logistic Regression (L_2 regularization, $C=1.0$), Random Forest (100 trees), Linear SVM, and Stratified Dummy baseline.
- Evaluation: Evaluated on 61 test samples held out from the 304 unique clinical presentations.
- **Verification Evidence**:
- `predict_disease.py` lines 131–158: `python pred_disease = self.model.predict(X_input)[0] probs = self.model.predict_proba(X_input)[0] top_indices = np.argsort(probs)[::-1][:top_k]`
- **Limitations & Discrepancies**:
- **No Similarity Metric**: The system computes **no Jaccard index, Cosine similarity, or semantic overlap**. It evaluates a discriminant hyperplane separating 41 closed classes.
- **Closed-World Assumption**: When given an emerging disease symptom set (e.g., novel respiratory-hemorrhagic syndrome), the model forces the input into one of 41 known training labels.
- **No Feature Overlap / Difference Reporting**: The CLI does not report which symptoms match a reference profile and which symptoms diverge.
- **Stage 1 Status**: **PARTIALLY IMPLEMENTED (Implemented as a closed-world classifier, NOT an open-world similarity engine)**.

5.2 Stage 2: Therapeutic Evidence Investigation & Entity Linkage

- **Intended Requirement**: Link identified similar diseases to existing treatments, active pharmaceutical ingredients, biological targets, compound bioactivity, and clinical development phases.
- **Actual Implementation**:
- Implemented in `src/models/prioritize_therapeutic_candidates.py`.
- Merges `therapeutic_associations.csv` (Open Targets) with `chembl_activity.csv` (ChEMBL) on `target_symbol`, and with `chembl_molecules.csv` on `molecule_chembl_id`.
- Evaluates Lipinski Rule of 5 and Veber rules for 6,319 molecules.
- Computes Composite Priority Score ($CPS \in [0.0, 1.0]$) across 25,650 candidates for 37 diseases.
- **Verification Evidence**:
- `prioritize_therapeutic_candidates.py` lines 207–218:

$$CPS = 0.35 \times S_{\text{target}} + 0.35 \times S_{\text{potency}} + 0.15 \times S_{\text{druglike}} + 0.15 \times S_{\text{evidence}}$$

- Top candidate output verified in `data/processed/therapeutic_candidate_rankings.csv`.
- **Limitations & Discrepancies**:
- **Offline Static Processing**: This stage runs as an offline batch process on a fixed list of 37 diseases.
- **Disconnected from Inference**: `src/inference/predict_disease.py` **never calls or loads** `therapeutic_candidate_rankings.csv`. When a user runs inference for "Malaria", the CLI displays Malaria and terminates without retrieving candidates.
- **Stage 2 Status**: **PARTIALLY IMPLEMENTED (Functional as an offline batch ranking pipeline; disconnected from the user-facing inference workflow)**.

5.3 Stage 3: Therapeutic-Development Timeline & Cartesian Join Analysis

- **Intended Requirement**: Analyze historical development timelines, clinical trial transitions, phase durations, and regulatory approval milestones.
- **Actual Implementation**:
- Implemented in `src/data_processing/build_therapeutic_timeline.py` and `src/analysis/validate_therapeutic_timeline.py`.
- Harmonizes 12,243 Drugs@FDA product approvals with ChEMBL indication records.
- Performs 5-tier entity matching (3,059 of 3,142 ingredients matched, 97.36%).
- Identifies that the 555,323 master timeline records result from an 18.25x Cartesian expansion across formulation, strength, and packaging lines for 30,430 unique triplets.
- Classifies records into Phase 4 verified approved indications (72,277 records, 13.02%) vs. Phase 1–3 investigational clinical trials (483,043 records, 86.98%).
- **Verification Evidence**:
- Verified in `outputs/timeline_analysis/timeline_quality_report.txt` and `mapping_audit_summary.json`.
- **Limitations & Discrepancies**:
- **No Longitudinal Candidate Tracking**: The dataset records the molecule's earliest FDA approval year (`first_approval_year`) and the ChEMBL maximum indication phase (`max_phase_for_ind`). It does **not** contain historical chronological transition timestamps (e.g., date entered Phase 1, date transitioned to Phase 2, date IND filed).
- Therefore, empirical phase-to-phase transition durations cannot be extracted from this dataset.
- **Stage 3 Status**: **PARTIALLY IMPLEMENTED (High-quality regulatory audit and Cartesian disambiguation; lacks chronological transition tracking)**.

5.4 Stage 4: Estimation of Medication Availability Future Year

- **Intended Requirement**: Use symptom similarity, therapeutic evidence, and developmental timelines to estimate a future calendar year when medication for the emerging disease might become available.
- **Actual Implementation**:
- Implemented in `src/analysis/forecast_therapeutic_timeline.py`.
- Script evaluates the 37 pre-compiled diseases in `therapeutic_candidate_rankings.csv`.
- Reads `max_clinical_phase` per disease from ChEMBL indications.
- Adds fixed benchmark durations from literature (DiMasi et al., 2003, 2016):
  - Phase 1: 2.1 years
  - Phase 2: 3.2 years
  - Phase 3: 3.8 years
  - FDA NDA Review: 1.5 years
- Formula: $\hat{Y} = 2026 + \sum_{p \ge P_{\text{current}}} \tau_p$.
- Generates `outputs/timeline_analysis/experimental_forecast_projections.csv`.
- **Verification Evidence**:
- Inspecting `experimental_forecast_projections.csv`:
  - 33 diseases (AIDS, Diabetes, Malaria, Hypertension, etc.) have Phase 4 drugs $\to$ Projected Milestone Year = **2028** (2026 + 1.5y review).
  - Typhoid (Phase 3) $\to$ **2031**.
  - Impetigo (Phase 2) $\to$ **2034**.
  - Cervical Spondylosis & Paralysis (Phase 0) $\to$ **2041** (4.5y discovery + 10.6y clinical).
- **Limitations & Discrepancies**:
- **No Machine Learning Model**: This is an arithmetic lookup table, not a trained predictive machine learning model.
- **No Emerging Disease Capability**: Only evaluates the 37 known diseases.
- **Not Integrated with Symptoms**: Disconnected from `predict_disease.py`.

- **Misleading Milestone Meaning**: For 34 of 37 diseases, the milestone was historically achieved decades ago. Labeling 2028 as the milestone year for Malaria or AIDS represents a hypothetical label-expansion assumption, not the discovery of a first-in-class treatment.
- **Stage 4 Status**: **PARTIALLY IMPLEMENTED (Deterministic prototype exists for known diseases; not implemented for emerging diseases or integrated with inference)**.

5.5 Stage 5: Modeling Advances in Medical & Technological Research

- **Intended Requirement**: Computationally account for evolving biotechnology, AI-assisted drug discovery, and clinical trial efficiency over time.
- **Actual Implementation**:
- Scanned all source files (`src/**/*.py`) and configuration files for time-dependent trend functions, acceleration coefficients, or technology investment parameters.
- **Zero mathematical formulas or code routines exist** that model technological acceleration.
- The DiMasi benchmarks utilized are static constants published in 2003 and 2016.
- **Verification Evidence**:
- In `docs/future_therapeutic_forecast_limitations.md` (lines 55–80), technological progress is discussed conceptually as an unquantified epistemic limitation.
- **Limitations & Discrepancies**:
- Technological progress is acknowledged in documentation, but is **absent from the computational pipeline**.
- **Stage 5 Status**: **NOT IMPLEMENTED (Conceptual discussion only)**.

Section 6: End-to-End Integration Audit

A critical software architecture audit was conducted to determine whether data and control flows connect the intended stages into an end-to-end pipeline.

Intended Pipeline Transition	Implemented Source Module & Mechanism	Downstream Consumer / Destination	Actual Integration Status	Evidence & Defect Analysis
Emerging Disease Symptoms $\to$ Symptom Representation	`DiseasePredictor.vectorize_symptoms` (`src/inference/predict_disease.py`)	Vector reshaped to $(1, 132)$ for model input	**VERIFIED**	Successfully parses comma-separated strings, normalizes whitespace and casing, and aligns against 132 features.
Symptom Vector $\to$ Similarity Calculation	`None` (Calls `self.model.predict` and `predict_proba`)	Closed-class argmax decision	**NOT IMPLEMENTED**	**Broken Transition**: Bypasses similarity metrics entirely; forces input through a 41-class classification hyperplane.
Similarity Metrics $\to$ Reference Disease Ranking	`np.argsort(probs)` on classifier softmax margins	`top_candidates` in CLI output	**PARTIALLY IMPLEMENTED**	Sorts classes by model posterior probability, not by feature overlap or metric distance.
Ranked Disease $\to$ Therapeutic Evidence Retrieval	`None` (`predict_disease.py` terminates after printing disease)	Disconnected from `therapeutic_candidate_rankings.csv`	**DISCONNECTED**	**Critical Architectural Gap**: CLI output does not link to the candidate prioritization database.
Therapeutic Candidates $\to$ Target & Bioactivity Linkage	`prioritize_candidates()` (`src/models/prioritize_therapeutic_candidates.py`)	`therapeutic_candidate_rankings.csv`	**VERIFIED (BATCH ONLY)**	Inner joins Open Targets, ChEMBL activity, and ChEMBL molecules. Runs offline for 37 fixed diseases.
Therapeutic Candidates $\to$ Developmental Timeline Linkage	`None` (`forecast_therapeutic_timeline.py` reads `max_phase` , not timeline rows)	Disconnected from `therapeutic_development_timeline.csv`	**DISCONNECTED**	**Broken Transition**: The 555k master timeline dataset is never queried by the forecasting module.
Development History $\to$ Future-Year Forecast Calculation	`compute_projected_timeline()` (`forecast_therapeutic_timeline.py`)	`experimental_forecast_projections.csv`	**PARTIALLY IMPLEMENTED**	Adds fixed DiMasi benchmark constants to 2026. Operates offline on 37 pre-compiled diseases.
Forecast Calculation $\to$ User-Facing Presentation	`None`	Terminal execution of `forecast_therapeutic_timeline.py`	**DISCONNECTED**	CLI inference user never sees a projected year or milestone decision window.

Integration Summary Verdict

The end-to-end workflow does NOT exist as a connected pipeline. The repository consists of isolated functional islands: an inference CLI island, a batch candidate-ranking island, an epidemiological analysis island, an FDA timeline validation island, and an offline milestone projection island.

Section 7: Machine Learning and Scientific Validation

7.1 Classification Benchmark vs. Clinical Realities

- **Reported Metric**: 100% Accuracy, 1.0000 Macro F1 on held-out test data ($N=61$).
- **Scientific Audit Finding**:
- In `data/raw/symptom_disease.csv`, there were 4,961 rows representing only 304 unique clinical symptom patterns (93.87% synthetic duplication rate).
- The authors properly deduplicated this to 304 unique patterns (`data/processed/symptom_disease_clean.csv`) and enforced a stratified 80/20 train/test split with zero pattern overlap.
- The 100% test accuracy is mathematically legitimate on this benchmark because the canonical symptom vectors for these 41 diseases are linearly separable.
- **Scientific Caveat**: This metric reflects algorithmic correctness on a synthetic, noise-free benchmark. It does **not** indicate clinical diagnostic infallibility in human patients, where symptoms are probabilistic, noisy, and comorbid.

7.2 Disease-to-Medication Mapping Specificity

- **Audit Finding**:
- The candidate prioritization engine merges Open Targets gene associations with ChEMBL bioactivity on `target_symbol`.
- This is an **indirect target-mediated association**, not a disease-specific clinical indication!
- For example, if Open Targets associates Gene G with Disease D, and ChEMBL has an assay showing Molecule M binds to Gene G, the pipeline generates a candidate triplet (D, G, M).
- Molecule M may have been developed for an entirely different pathology (e.g., oncology) that happens to share target G.
- The pipeline does not claim Molecule M is approved for Disease D, but users must understand that high-ranking candidates are in-silico target hypotheses, not proven disease treatments.

7.3 Candidate Prioritization & Negative Control Analysis

- **Formula**:

$$CPS = 0.35 \times S_{\text{target}} + 0.35 \times S_{\text{potency}} + 0.15 \times S_{\text{druglike}} + 0.15 \times S_{\text{evidence}}$$

- **Negative Control Audit (Medroxyprogesterone Acetate / AIDS / PGR)**:
- In `data/processed/therapeutic_candidate_rankings.csv`, the #1 ranked candidate for **AIDS** is **MEDROXYPROGESTERONE ACETATE** ($CPS = 0.7689$).
- Medroxyprogesterone is a synthetic progestin used in female contraception. It has no direct anti-retroviral efficacy against HIV.
- Why did it rank #1?
  1. Open Targets lists an association (0.7423) between AIDS and the Progesterone Receptor (PGR) due to literature co-occurrence in studies examining hormonal contraception and HIV transmission risk.
  2. Medroxyprogesterone binds to PGR with nanomolar potency ($p\text{ChEMBL} = 9.77 \to S_{\text{potency}} = 0.96$).
  3. It exhibits complete Lipinski compliance ($S_{\text{druglike}} = 1.00$).
  4. It is an FDA-approved Phase 4 drug ($S_{\text{evidence}} = 1.00$).
- **Verdict**: This finding is a **vital negative control**. It conclusively proves that CPS is an uncalibrated multi-criteria screening heuristic, **not** clinical proof of efficacy.

7.4 Epidemiological Time-Series Benchmarking

- **Audit Finding**:
- Evaluated on a 10-year held-out horizon (2015–2024) for WHO global measles surveillance:
  - **Naive Persistence Baseline ($\hat{y}_t = y_{t-1}$)**: $\text{MAE} = 53,025$, $\text{MAPE} = 91.95\%$.
  - **3-Year Moving Average**: $\text{MAE} = 57,401$, $\text{MAPE} = 105.93\%$.
  - **Autoregressive Ridge (Lags + Trend)**: $\text{MAE} = 155,157$, $\text{MAPE} = 240.19\%$.
- **Scientific Integrity**: The persistence baseline vastly outperformed the complex autoregressive model because global epidemiological surveillance series exhibit severe non-stationarity due to policy changes, conflicts, and reporting shifts. The project honestly documented this negative result rather than concealing it.

Section 8: Future-Year Forecasting Audit

This section directly answers the ten mandatory audit questions regarding future-year estimation.

1. Does the project produce a future year?

Yes, but only in an offline batch output file (`outputs/timeline_analysis/experimental_forecast_projections.csv`), **not** in the user-facing inference interface (`src/inference/predict_disease.py`).

2. What does that year actually represent?

It represents a **hypothetical regulatory decision window** for an existing clinical candidate compound to complete remaining clinical trial phases. It does **not** represent the discovery of a cure for an emerging disease.

3. Is it generated by a model, a formula, historical averages, a fixed assumption, or a combination?

It is generated by a **deterministic formula using fixed historical industry averages** (DiMasi et al., 2003, 2016). No machine learning model is trained, fitted, or evaluated.

4. Is the estimate linked to a newly emerging disease?

No. The forecasting script operates exclusively on a fixed array of 37 existing disease categories present in `therapeutic_candidate_rankings.csv`.

5. Does the calculation use the closest known diseases identified by symptom similarity?

No. The forecasting routine has no connection to the symptom vectorizer, symptom classifier, or any similarity metric.

6. Does it use relevant treatment and development evidence?

Partially. It extracts the maximum observed clinical phase (`max_phase`) from ChEMBL indications for that disease, but does not use molecular structures, target mechanisms, or timeline transition histories.

7. Does it account for medical and technological progress?

No. The phase durations used are static constants from 2003/2016 literature.

8. Is the method validated against historical outcomes?

No. The forecasting implementation has not undergone chronological backtesting against historical FDA approvals.

9. Does it report uncertainty?

Yes. It provides an empirical heuristic uncertainty window ($[0.65 \times \tau_{\text{cum}}, 1.45 \times \tau_{\text{cum}}]$) and cumulative transition success probability (P_{success}).

10. What claims about the future year are scientifically justified?

The only justified claim is: "*Under aggregate industry averages, if an active candidate in Phase P advances without attrition, its median regulatory review decision would occur within $[Y_{\min}, Y_{\max}]$.*" Any claim of predicting when an emerging disease will be cured is scientifically unjustified.

Section 9: Testing and Reproducibility Audit

The test suite was executed in the production environment:

```
pytest -v
```

Execution Log & Test Verification

```
 tests/test_epidemiological_analysis.py::test_epidemiological_trends_dataset PASSED [  3%]
tests/test_epidemiological_analysis.py::test_missing_value_retention PASSED [  7%]
tests/test_epidemiological_analysis.py::test_outbreak_surge_detection_logic PASSED [ 11%]
tests/test_symptom_classifier.py::test_raw_symptom_dataset_schema PASSED [ 15%]
tests/test_symptom_classifier.py::test_deduplication_and_unique_patterns PASSED [ 19%]
tests/test_symptom_classifier.py::test_leakage_free_train_test_split PASSED [ 23%]
tests/test_symptom_classifier.py::test_persisted_model_reproducibility PASSED [ 26%]
tests/test_symptom_classifier.py::test_inference_predictor_api PASSED    [ 30%]
tests/test_therapeutic_prioritization.py::test_input_schemas_and_identifiers PASSED [ 34%]
tests/test_therapeutic_prioritization.py::test_druglikeness_scoring_logic PASSED [ 38%]
tests/test_therapeutic_prioritization.py::test_clinical_evidence_bonus_logic PASSED [ 42%]
tests/test_therapeutic_prioritization.py::test_candidate_rankings_output_integrity PASSED [ 46%]
tests/test_timeline_validation.py::test_fda_timeline_schema PASSED       [ 50%]
tests/test_timeline_validation.py::test_fda_chembl_matching_rate PASSED  [ 53%]
tests/test_timeline_validation.py::test_evidence_classification_audit PASSED [ 57%]
tests/test_unit_and_pipeline_fixtures.py::test_symptom_vectorizer_edge_cases PASSED [ 61%]
tests/test_unit_and_pipeline_fixtures.py::test_feature_consistency_between_training_and_inference PASSED [ 65%]
tests/test_unit_and_pipeline_fixtures.py::test_druglikeness_exact_boundaries PASSED [ 69%]
tests/test_unit_and_pipeline_fixtures.py::test_clinical_evidence_bonus_comprehensive PASSED [ 73%]
tests/test_unit_and_pipeline_fixtures.py::test_composite_priority_score_mathematical_weights PASSED [ 76%]
tests/test_unit_and_pipeline_fixtures.py::test_drug_normalization_and_salt_stripping PASSED [ 80%]
tests/test_unit_and_pipeline_fixtures.py::test_milestone_forecast_durations_and_success_rates PASSED [ 84%]
tests/test_unit_and_pipeline_fixtures.py::test_epidemiological_metric_zero_denominator_protection PASSED [ 88%]
tests/test_unit_and_pipeline_fixtures.py::test_surge_detector_flat_series_no_false_alarm PASSED [ 92%]
tests/test_unit_and_pipeline_fixtures.py::test_negative_control_medroxyprogesterone_audit PASSED [ 96%]
tests/test_unit_and_pipeline_fixtures.py::test_lightweight_prioritization_mock PASSED [100%]
============================== 26 passed in 9.34s ==============================
```

- **Environment**: Python 3.12.10, `pytest` 9.1.1, Windows 11.
- **Reproducibility Assessment**: **HIGH**. All 26 automated tests pass cleanly with zero warnings or failures. The software engineering baseline is robust.

Section 10: Requirements Achievement Scorecard

ID	Requirement	Evidence from Implementation	Status	Confidence	Identified Gap
REQ-1	Accept symptom profile of newly emerging disease	`DiseasePredictor.vectorize_symptoms` accepts symptom lists/strings.	**PARTIAL**	High	Normalizes symptoms, but maps only to a closed 132-feature vocabulary; no open-vocabulary or ontology mapping.
REQ-2	Calculate degree of symptom similarity to known diseases	`predict_disease.py` lines 135–158.	**FAIL**	High	Uses a closed-class classifier decision boundary ($P(y\|x)$), not an explicit distance/similarity metric.
REQ-3	Identify overlapping and differing symptoms	`predict_disease.py` lines 117–129.	**FAIL**	High	Reports matched vs. unmatched user inputs, but does not compare against reference disease profiles.
REQ-4	Retrieve treatment evidence for similar diseases	`prioritize_therapeutic_candidates.py` ranks 25,650 candidates.	**PARTIAL**	High	Candidate ranking exists as an offline batch table, but is completely disconnected from the inference CLI.
REQ-5	Link treatments to biological targets & compound bioactivity	`therapeutic_associations.csv` merged with ChEMBL activity & molecules.	**PASS**	High	Verified inner join on `target_symbol` and `molecule_chembl_id` with median $p\text{ChEMBL}$ calculation.
REQ-6	Extract historical therapeutic development timelines	`therapeutic_development_timeline.csv` (555,323 rows).	**PARTIAL**	High	Integrates FDA approvals and ChEMBL indications, but lacks chronological transition timestamps.
REQ-7	Model historical phase durations & transition latencies	`forecast_therapeutic_timeline.py` lines 48–55.	**PARTIAL**	High	Uses static literature constants (DiMasi et al.); does not learn durations from historical timeline data.
REQ-8	Estimate future year for medication availability	`experimental_forecast_projections.csv` projects years for 37 diseases.	**PARTIAL**	High	Outputs milestone years for known diseases via formula, but cannot predict for an emerging disease.
REQ-9	Computationally account for advances in medical/AI research	Scanned all codebase files.	**FAIL**	High	Zero implementation of technological trend models or dynamic acceleration parameters.
REQ-10	Validate forecast against historical development outcomes	Checked test and evaluation suites.	**FAIL**	High	No chronological backtesting or historical milestone evaluation routines implemented.
REQ-11	Provide end-to-end connected execution workflow	Traced `run_theralens.py` and `predict_disease.py`.	**FAIL**	High	Pipeline stages execute as independent silos without passing data to form an end-to-end forecast.

Section 11: Prioritized Gap Analysis

Gap 1 (Critical): Absence of an Open-World Symptom Similarity Engine

- **Description**: The project relies on a 41-class closed-world classifier instead of an open-world similarity algorithm.
- **Why It Matters**: An emerging disease, by definition, is not in the 41 training classes. Forcing it into a known class generates invalid differential diagnoses.
- **Required Implementation**: Implement explicit metric similarity (Jaccard, Cosine, or weighted Ontology overlap using Human Phenotype Ontology / MeSH) between the input vector and all reference disease symptom centroids. Output top-k similar diseases with explicit overlap percentages.
- **Modules Requiring Changes**: `src/models/symptom_similarity.py` (new), `src/inference/predict_disease.py`.
- **Priority**: **CRITICAL**.

Gap 2 (Critical): Disconnection Between Symptom Inference and Therapeutic Evidence

- **Description**: The inference CLI outputs a disease label and stops. It never queries the 25,650 ranked candidates in `therapeutic_candidate_rankings.csv`.
- **Why It Matters**: The user cannot see what therapeutic targets, compounds, or mechanisms exist for the similar diseases identified.
- **Required Implementation**: Connect the output of the similarity engine to an automated lookup in `therapeutic_candidate_rankings.csv`, returning the top biological targets and candidate molecules.
- **Modules Requiring Changes**: `src/inference/predict_disease.py`.
- **Priority**: **CRITICAL**.

Gap 3 (Critical): Disconnection Between Treatment Evidence and Future-Year Forecasting

- **Description**: The future-year forecasting script is an offline batch calculation on 37 diseases. It cannot be invoked dynamically for an emerging disease query.
- **Why It Matters**: The central objective—estimating when medication for an emerging disease might become available—cannot be performed interactively.
- **Required Implementation**: Create an integrated forecast function that takes the retrieved candidates for the similar disease(s) and computes projected milestone windows dynamically.
- **Modules Requiring Changes**: `src/analysis/forecast_therapeutic_timeline.py`, `src/inference/predict_disease.py`.
- **Priority**: **CRITICAL**.

Gap 4 (High): Lack of Machine Learning in Year Forecasting

- **Description**: Milestone years are calculated using simple addition of static literature constants, rather than a trained predictive model.
- **Why It Matters**: The project claims "Machine Learning-Based Forecasting", but machine learning is only used in the symptom classifier.
- **Required Implementation**: Train a regression or survival analysis model (e.g., Cox Proportional Hazards, Accelerated Failure Time, or Gradient Boosted Survival) on historical development transitions.
- **Modules Requiring Changes**: `src/analysis/forecast_therapeutic_timeline.py`.
- **Priority**: **HIGH**.

Gap 5 (Medium): Zero Computational Representation of Technological Progress

- **Description**: Technological and medical advancements are discussed only in text documentation.
- **Why It Matters**: Violates the original requirement to account for evolving medical research capabilities.
- **Required Implementation**: Define explicit, parameterized scenario multipliers (e.g., Conservative: $1.0\times$, Accelerated AI Discovery: $0.80\times$, Breakthrough: $0.60\times$ phase latency) and report bounds.
- **Modules Requiring Changes**: `src/analysis/forecast_therapeutic_timeline.py`.
- **Priority**: **MEDIUM**.

Section 12: Recommended Implementation Roadmap

To transition TheraLens from its current state of isolated analytical modules into a fully integrated research forecasting system, a five-phase implementation roadmap is recommended:

```
[ Phase 1: Similarity Engine ] ──> [ Phase 2: Dynamic Linkage ] ──> [ Phase 3: Survival Forecasting ]
  - Replace closed classifier        - Connect similarity output      - Replace static DiMasi sum
    with Jaccard/Cosine metric         to candidate rankings table      with survival regression model
  - Report overlap & difference      - Return targets & molecules     - Historical backtesting (pre/post 2015)
                                                                                  |
                                                                                  ▼
[ Phase 5: Unified Web / CLI ] <── [ Phase 4: Tech Acceleration ]
  - End-to-end execution               - Parameterized scenario bounds
  - Single command query               - AI-assisted trial efficiency
  - Comprehensive report card            multipliers
```

Phase 1: Open-World Symptom Similarity Engine (Est. Effort: 2 Days)

- Build `src/models/symptom_similarity.py`.
- Compute pairwise Cosine and Jaccard similarity between the input symptom set and reference symptom vectors across all 41 diseases.
- Output similarity percentage, list of shared symptoms, and list of differentiating symptoms.

Phase 2: Dynamic Evidence Retrieval Integration (Est. Effort: 1 Day)

- Modify `src/inference/predict_disease.py` to import `data/processed/therapeutic_candidate_rankings.csv` and `disease_target_rankings.csv`.
- When top similar diseases are identified, dynamically extract top biological targets, top compounds, median $p\text{ChEMBL}$ potencies, and current clinical advancement phases.

Phase 3: Longitudinal Transition Survival Model (Est. Effort: 3 Days)

- Reconstruct developmental intervals from historical clinical trials (e.g., ClinicalTrials.gov API).
- Train an Accelerated Failure Time (AFT) or survival model on trial durations conditioned on therapeutic area, modality (small molecule vs. biologic), and target class.
- Validate via chronological backtesting (train on pre-2015 trials, predict 2016–2025 approval years).

Phase 4: Technological Acceleration Scenarios (Est. Effort: 1 Day)

- Incorporate parameterized discovery acceleration factors ($\alpha_{\text{tech}} \in [0.70, 1.00]$) reflecting high-throughput computational screening.
- Report scenario-based decision windows (e.g., Standard Pipeline vs. AI-Accelerated Pipeline).

Phase 5: Unified CLI and End-to-End Report Generation (Est. Effort: 1 Day)

- Unify the complete pipeline under a single command: `powershell python src/inference/forecast_emerging_disease.py --symptoms "fever,cough,hemoptysis" --disease_name "Novel_Pathogen_X"`
- Generate a single, comprehensive markdown/PDF dossier detailing similarity, targets, candidate molecules, and projected milestone availability windows.

Section 13: Final Verdict & Core Question Answer

13.1 Official Audit Verdict

$$\mathbf{VERDICT: \; PARTIALLY \; ACHIEVED}$$

Justification:

1. **What is Successfully Built**: The project has successfully constructed high-quality, rigorously audited foundational components: a 132-symptom vocabulary with zero-leakage deduplication; a multi-criteria candidate prioritization engine ranking 25,650 candidates across 3,636 targets; an audited 555k-row regulatory timeline with Cartesian disambiguation and 97.36% FDA-to-ChEMBL entity resolution; and a 26-test automated test suite with 100% pass rate.
2. **Why the Central Objective is Incomplete**: The central research objective—connecting emerging disease symptoms to similar diseases, treatment evidence, and a predictive future medication availability year—**is broken into disconnected functional silos**. The system uses a closed-world classifier rather than an open-world similarity algorithm, the inference CLI terminates without retrieving treatment evidence, the future-year forecast is an offline arithmetic lookup rather than a learned predictive model, and technological progress is not computationally modeled.

13.2 Answer to the Core Research Question

> ***"If I enter the symptoms of a newly emerging disease today, can the current TheraLens application compare them with existing diseases, examine relevant treatment evidence, and generate a scientifically defensible estimate of the future year when medication might become available?"***

Direct Answer: NO

Concrete Scientific Justification:

1. **Cannot Compare Emerging Disease Symptoms via Similarity**: If you input symptoms of a newly emerging disease (e.g., a novel viral hemorrhagic fever), `src/inference/predict_disease.py` cannot compute metric similarity. Instead, its multinomial classifier forces the input into one of its 41 known training classes (e.g., predicting "Dengue" or "Malaria" with an uncalibrated softmax score). It cannot report similarity percentages or identify overlapping versus differing symptom features.
2. **Cannot Examine Treatment Evidence from the Inference Interface**: Once the classifier predicts a disease label, the program **terminates**. It does not query `therapeutic_candidate_rankings.csv`, does not display associated biological targets, and does not retrieve candidate molecules.
3. **Cannot Generate a Future Medication Availability Year for the Query**: The inference engine outputs **no year at all**. The only year-forecasting code in the repository (`src/analysis/forecast_therapeutic_timeline.py`) is an offline script that evaluates 37 pre-compiled known diseases using static arithmetic additions (outputting 2028 for 34 of them). It cannot accept your emerging disease input or calculate a dynamic availability year.

Summary: TheraLens has built the necessary biomedical data and software foundations, but it has not yet assembled them into the connected, end-to-end emerging disease forecasting tool originally envisioned.